

Fiche de révision — Monitoring, FinOps & Sécurité Azure (TP4)

Cours magistral « *Monitoring, FinOps et sécurité Azure* » — Bloc 4, Mastère Dev Manager Full Stack. Objectif : **piloter** une infrastructure Azure déjà déployée (observer, maîtriser les coûts, sécuriser, auditer, recommander). Cas fil rouge : **ShopEasy**.

1. De la construction au pilotage

Déployer n'est qu'une première étape : une plateforme cloud doit ensuite être **exploitée**. Objectifs de l'exploitation : **disponibilité, performance, sécurité, maintenabilité, maîtrise des coûts, traçabilité**.

⚠ Le cloud rend l'infra **dynamique** (créer/modifier/détruire vite) → souple mais aussi nouveaux risques : ressources oubliées, droits trop larges, dépenses inattendues, absence de supervision, faible traçabilité.

Cas entreprise. Un environnement peut « marcher » en apparence tout en accumulant des risques invisibles : coûts cachés, ports ouverts, droits excessifs, logs non collectés, aucune alerte. Le TP4 apprend à raisonner comme une **équipe d'exploitation** : observer, détecter, diagnostiquer, alerter, optimiser, sécuriser, documenter.

2. Monitoring / Observabilité / Audit / Sécurité

Quatre questions complémentaires (distinction d'examen classique) :

Concept	Question centrale	Exemple
Monitoring	<i>Est-ce que ça fonctionne ?</i>	CPU d'une VM dépasse 85 %
Observabilité	<i>Pourquoi ça ne fonctionne pas ?</i>	Les logs montrent une erreur de connexion à la base
Audit	<i>Qui a fait quoi ?</i>	Un utilisateur a modifié une règle réseau
Sécurité	<i>Le système est-il protégé ?</i>	Un port d'admin est exposé à Internet

Les 3 signaux classiques : - **Métriques** : valeurs numériques dans le temps (CPU, mémoire, latence, requêtes) → *indique qu'un problème existe*. - **Logs** : événements textuels détaillés → *aident à comprendre le contexte*. - **Traces** : suivi d'une requête à travers plusieurs composants → *chemin d'une requête en système distribué*.

Cas entreprise. Une métrique CPU à 90 % dit *qu'il y a un problème* ; les logs révèlent *pourquoi* (timeout base de données) ; la trace montre *où* la requête se bloque dans la chaîne de microservices. Les trois sont complémentaires.

3. Azure Monitor : composants

Service central de supervision : il **collecte** → **analyse** → **déclenche des actions** sur les ressources Azure.

Composant	Rôle
Metrics	Stockage/analyse de métriques numériques dans le temps
Logs	Centralisation/analyse de logs dans un workspace
Log Analytics Workspace	Espace pour stocker, interroger (KQL) et corréler les logs
Alert Rules	Règles déclenchées quand une condition est atteinte
Action Groups	Destinataires/actions associés à une alerte (email, SMS, webhook...)
Workbooks	Rapports interactifs (texte + visualisations + requêtes)
Dashboards	Tableaux de bord synthétiques dans le portail
Activity Log	Journal des opérations de gestion sur les ressources

Métriques vs Logs :

	Métriques	Logs
Nature	Numériques agrégées	Événements détaillés
Usage	Alerte rapide sur seuil	Diagnostic / investigation
Coût	Plus prévisible	Dépend du volume ingéré/conservé

Cas entreprise. On centralise tous les logs des VM, du Load Balancer et de la base dans **un Log Analytics Workspace commun**. En cas d'incident, une seule requête KQL corréle les événements de toutes les couches au lieu de fouiller ressource par ressource.

4. Construire une stratégie de monitoring

⚠ Piège : vouloir **tout surveiller** sans priorisation. On part des **objectifs métier/opérationnels**.

La question avant de créer une alerte : *si elle se déclenche, quelqu'un doit-il vraiment agir ?* Si non → c'est une info de **dashboard**, pas une alerte.

Indicateurs essentiels par axe : Disponibilité (état VM, taux d'erreur HTTP) · Performance (CPU/mémoire/disque/latence) · Capacité (stockage, connexions) · Sécurité (connexions suspectes, changements IAM, ports exposés) · Coût (par service/tag, budget consommé) · Qualité d'exploitation (nb d'incidents, temps de résolution).

SLI / SLO / SLA (à ne pas confondre) : - **SLI** (*Indicator*) = indicateur **mesuré** (ex. taux de disponibilité réel). - **SLO** (*Objective*) = objectif **interne** sur l'indicateur (ex. 99,5 %/mois). - **SLA** (*Agreement*) = engagement **contractuel** avec un client (avec pénalités).

Cas entreprise. Un SaaS s'engage sur un **SLA** de 99,9 % auprès de ses clients. En interne il vise un **SLO** plus strict (99,95 %) mesuré par un **SLI** (disponibilité HTTP), pour garder une marge avant de violer le contrat.

5. Alertes & gestion des incidents

Une alerte doit **transformer une observation en action** : condition + période + **sévérité** + **action group** + **runbook** (procédure).

⚠ **Fatigue d'alerte** : trop d'alertes tue l'alerte (l'équipe finit par les ignorer). Distinguer : **critiques** (action immédiate) / **avertissements** (analyse) / **infos** (reporting seul).

Cycle de gestion d'incident : Détecter → Qualifier → Diagnostiquer → Corriger → **Capitaliser**.

Cas entreprise. Une équipe reçoit 200 alertes/jour, toutes au même niveau → elle les ignore et rate l'incident critique. La correction : ne garder en alerte que l'**actionnable**, mettre le reste en dashboard, et associer chaque alerte critique à un runbook (*quoi vérifier, qui contacter, quelle action*).

6. FinOps sur Azure

FinOps = finance + technologie + opérations pour **comprendre, piloter et optimiser** la dépense cloud. Ce n'est pas « réduire les coûts » mais **maximiser la valeur par rapport au coût**.

Pourquoi le cloud change la donne : les coûts apparaissent **au fil de la consommation** (et non à l'achat), et varient vite selon l'activité, les erreurs de config ou les ressources oubliées.

Les 3 grands objectifs : 1) rendre les coûts **visibles** · 2) **responsabiliser** les équipes (coût ↔ propriétaire) · 3) **optimiser en continu**.

Outil Azure	Rôle
Cost Management	Analyse, suivi, optimisation des coûts
Budgets	Seuils financiers + notifications
Cost alerts	Alerte sur dépassement/anomalie
Pricing Calculator	Estimation avant déploiement
Tags	Affecter les coûts par projet/app/équipe/env
Reservations / Savings Plans	Réduction pour usages prévisibles/stables
Advisor	Recommandations coût/sécurité/perf

Axes d'optimisation : supprimer les ressources inutilisées, éteindre les env hors-prod, redimensionner les VM, bon niveau de service stockage/base, réservations si usage stable, budgets+alertes, **tags obligatoires**.

Cas entreprise. Une bonne reco FinOps ne dit pas « réduire les coûts ». Elle précise : *la ressource concernée, le problème, l'impact estimé, le risque, l'action proposée*. Ex. « La VM `vm-web-02` est à 5 % de CPU moyen → la passer de D4s à B2s : ~120 €/mois économisés, sans impact perf. »

7. Sécurité : principes fondamentaux

Modèle de responsabilité partagée :

Responsable	Exemples
Fournisseur (Azure)	Datacenters, matériel, hyperviseur, dispo des services
Client	Comptes, rôles, données, config réseau, chiffrement, journalisation, gouvernance
Partagée	Mises à jour, supervision, continuité selon le service

Identité & accès : Utilisateur → Groupe → **Rôle RBAC** → appliqué sur un **Scope** (management group → subscription → resource group → resource).

Moindre privilège : accorder **uniquement** les droits nécessaires → limite l'impact d'une erreur ou d'un compte compromis. ⚠ Donner **Owner par facilité** est une mauvaise pratique (Contributor ou Reader suffisent souvent).

Sécurité réseau (NSG) : fermer les ports inutiles, limiter SSH/RDP à des IP autorisées, **isoler les bases** (jamais exposées à Internet), segmenter les subnets, journaliser les changements.

Cas entreprise. Un compte de service de l'app a le rôle **Owner** sur toute la subscription « pour que ça marche ». S'il est compromis, l'attaquant contrôle tout. Le moindre privilège : un rôle custom limité au seul resource group de l'app.

8. Microsoft Defender for Cloud

Outil de **pilotage de la posture de sécurité** (pas juste un scanner) : - **Secure Score** : indicateur synthétique de posture. - **Recommendations** : actions pour corriger les faiblesses. - **Regulatory Compliance** : suivi de conformité (cadres de référence). - **Workload Protection** : protections avancées selon les plans activés.

Lecture critique : toutes les recommandations ne se valent pas → prioriser selon exposition, sensibilité des données, coût de correction, impact opérationnel, conformité, criticité métier.

Cas entreprise. Defender liste 80 recommandations. La DSI n'attend **pas** une liste brute mais une **priorisation** : « corriger d'abord les 3 ports d'admin exposés (risque de compromission), puis activer le chiffrement, le reste en backlog. »

9. Audit, traçabilité & preuves

Auditer répond à : *qui a modifié quoi ? quand ? quelle action a causé l'incident ? les ressources critiques sont-elles journalisées ?*

Source de traces	Utilité
Activity Log	Opérations de gestion sur les ressources
Resource Logs	Logs spécifiques de certains services
Entra ID logs	Connexions, identités, activités des comptes
Diagnostic settings	Export des logs vers Log Analytics / Storage / Event Hub
Defender for Cloud	Alertes et recommandations sécurité
Azure Policy	Conformité des ressources aux règles

Notion de preuve : en pro, il ne suffit pas de *dire* qu'une ressource est sécurisée, il faut le **démontrer** (capture de config, export de logs, dashboard, rapport d'audit, historique, règle Policy, preuve de revue d'accès).

Cas entreprise. Lors d'un audit ISO, l'auditeur demande « prouvez que seules 3 personnes ont accès à la prod et que tout changement est tracé ». Réponse : export RBAC + Activity Log centralisé dans Log Analytics. *La preuve, pas la parole.*

10. Gouvernance Azure

Éviter que la flexibilité du cloud ne devienne du désordre : règles de nommage, tagging, sécurité, coût, conformité.

Niveaux : Management Group (plusieurs subscriptions) → Subscription → Resource Group → **Tags** → **Azure Policy** (imposer/auditer des règles) → **RBAC** (autorisations).

Politique de tags recommandée : **Environment**, **Owner**, **Application**, **CostCenter**, **Criticality**, **DataSensitivity** (public/internal/confidential).

Cas entreprise. Une **Azure Policy** refuse automatiquement la création de toute ressource sans tags **Owner** et **CostCenter**. Résultat : impossible de créer une ressource « orpheline » → la gouvernance FinOps est appliquée à la source, pas a posteriori.

11. Analyse de risques (matrice risque / impact)

Identifier les scénarios défavorables, estimer probabilité × impact, proposer une remédiation.

Risque	Impact	Cause	Mesure corrective
VM exposée en SSH	Compromission serveur	NSG trop permissif	Restriction IP, Bastion, MFA

Risque	Impact	Cause	Mesure corrective
Absence de logs	Diagnostic impossible	Diagnostic settings absents	Centraliser dans Log Analytics
Coût inattendu	Dépassement budget	Ressources oubliées	Budget, tags, alertes, suppression
Droits excessifs	Action non autorisée	Owner trop large	RBAC minimal + revue d'accès
Stockage non protégé	Fuite/perte de données	Config faible	Chiffrement, accès privé, sauvegarde

Priorisation : Haute (compromission, indispo majeure, coût imminent) · Moyenne (nécessaire sans urgence) · Basse (optimisation non bloquante).

Cas entreprise. On ne corrige pas tout en même temps : un port d'admin ouvert (priorité **haute** = compromission possible) passe avant un tag manquant (priorité **basse**). La matrice oriente l'ordre des actions.

12. Recommandations pour une DSI

Une DSI n'attend pas une liste de services, mais une **décision argumentée, lisible et priorisée**, reliant **technique ↔ risque ↔ coût ↔ valeur métier**.

Structure d'une note : 1) Contexte · 2) Constats · 3) Risques · 4) Recommandations (priorité + justification) · 5) Plan d'action (court/moyen/long terme) · 6) Indicateurs de suivi.

Domaine	Recommandation	Priorité	Impact
Monitoring	Centraliser logs/métriques dans un workspace	Haute	Diagnostic plus rapide
FinOps	Rendre Environment et Owner obligatoires	Haute	Attribution des coûts
Sécurité	Restreindre les ports d'administration	Haute	Réduction surface d'attaque
Gouvernance	Budgets par environnement	Moyenne	Maîtrise financière
Audit	Exporter Activity Log vers Log Analytics	Moyenne	Traçabilité

Cas entreprise. Une note DSI efficace tient en 2 pages : elle dit *quoi faire, dans quel ordre, pour quel gain et quel risque évité* — pas un dump technique de 40 recommandations Defender.

Les réflexes à retenir pour le TP4

1. **Monitoring** (ça marche ?) ≠ **observabilité** (pourquoi ?) ≠ **audit** (qui ?) ≠ **sécurité** (protégé ?).
2. **Métriques** = **alerte sur seuil** ; **logs** = **diagnostic**.
3. **Une alerte non actionnable = du bruit** (sinon → dashboard).
4. **SLI** (mesuré) < **SLO** (objectif interne) < **SLA** (contrat).
5. **FinOps** = **valeur/coût**, pas juste réduction ; tags + budgets + Advisor.

6. **Moindre privilège** : jamais Owner par facilité.
7. **Defender** : prioriser les recommandations, pas tout corriger en vrac.
8. **Prouver** la sécurité (logs, Activity Log, Policy), ne pas l'affirmer.
9. **Note DSI** : contexte → constats → risques → recos prioritées → plan → indicateurs.

Glossaire express

Terme	Définition
Azure Monitor	Supervision : collecte/analyse métriques, logs, alertes
Log Analytics Workspace	Espace centralisé pour stocker/interroger les logs (KQL)
Activity Log	Journal des opérations de gestion sur les ressources
Action Group	Destinataires/actions déclenchés par une alerte
SLI / SLO / SLA	Indicateur mesuré / objectif interne / engagement contractuel
FinOps	Pilotage et optimisation des coûts cloud
Budget	Seuil financier de suivi de consommation
RBAC	Contrôle d'accès basé sur les rôles
Defender for Cloud	Posture de sécurité + protection des workloads
Secure Score	Indicateur synthétique de posture sécurité
Azure Policy	Imposer/auditer des règles de gouvernance

Note déploiement (Azure for Students). Comme aux TP précédents : `swedencentral` (et non `francecentral`), taille VM `Standard_B2ts_v2` si tu recrées des ressources.